CSE 344 – System Programming

# Homework #4

***Multi-Process & Multi-Threaded Concurrent Log Analysis Pipeline***

**Due Date: (02.04.2026) – 23:59**

| Important Rules |
| --- |
| This assignment must be implemented in C on a POSIX-compatible system and must compile and run on Debian 12 (64-bit) in VirtualBox. |
| Identical or suspiciously similar submissions will receive -100 points and will be reported for academic misconduct. |
| A report with screenshots is mandatory. Missing report = 0 points on the entire homework. |
| Late submissions are NOT accepted under any circumstances. |
| Post all questions in the Teams assignment thread so every student benefits from the answers. |

## 1. Overview

In this homework you will design and implement a Concurrent Log Analysis Pipeline. The system reads multiple log files, classifies log entries by severity level, searches for multiple user-specified keywords with level-weighted scoring, identifies high-priority sources through a separate Dispatcher stage, and produces a structured summary report.

The system must use both multi-process architecture (fork, shared memory, semaphores) and POSIX multi-threading (pthreads, mutexes, condition variables, barriers, thread-local storage) within each process. Every synchronization primitive taught in the course must appear in this assignment in a meaningful, non-trivial role.

This homework directly extends Homework #3 (processes, shared memory, IPC) and introduces pthreads as a new in-process concurrency mechanism. The two paradigms must coexist correctly in the same running system.

## 2. What Makes This Assignment Challenging

Compared to a simple producer-consumer exercise, this system includes the following additional challenges:

- Multiple keywords with level-weighted scoring (not just substring search).
- A Dispatcher Process stage between Reader and Analyzer processes that performs source-based priority filtering.

- A Watchdog Thread inside the parent process that monitors all child processes via pipes and prints live progress every 3 seconds.
- Thread-local storage (pthread_key_t) required for per-thread keyword accumulation inside Analyzer worker threads.
- Timed blocking (pthread_cond_timedwait) with configurable timeout in the Dispatcher and Aggregator stages.
- Per-source statistics: identifying which SOURCE fields generated the most weighted keyword hits.
- A binary-format checkpoint file written atomically by the Aggregator, in addition to the human-readable output file.

> ⚠ **Do not start writing code immediately. This system has seven process/thread types interacting through four shared memory regions. Design everything on paper first, then implement incrementally and test at each stage.**

# 3. System Scenario

Imagine a distributed server cluster producing multiple log files simultaneously. Your system must:

- Read all log files concurrently, each by a dedicated Reader Process.
- Route log entries through a central Dispatcher Process that classifies entries as normal or high-priority based on their SOURCE field.
- Analyze entries concurrently inside four Analyzer Processes (one per severity level), each using an internal thread pool.
- Aggregate all results in a single Aggregator Process, write two output files (text + binary), and post the final summary to the parent.
- Provide a live progress feed to the terminal every 3 seconds via a Watchdog Thread running inside the parent process.

# 4. Log File Format

Each log line follows exactly this format:

```
[YYYY-MM-DD HH:MM:SS] [LEVEL] [SOURCE] MESSAGE
```

- TIMESTAMP: datetime in YYYY-MM-DD HH:MM:SS, enclosed in [ ].
- LEVEL: one of ERROR, WARN, INFO, or DEBUG, enclosed in [ ]. Any line whose LEVEL field is not one of these four values must be silently skipped.
- SOURCE: an alphanumeric identifier (no spaces, max 63 characters), enclosed in [ ].
- MESSAGE: the remainder of the line after the closing ] of SOURCE (may contain spaces, digits, punctuation). Leading whitespace after the ] must be stripped.

Example log file:

```
[2025-03-10 08:15:02] [ERROR] [kernel]  Out of memory: kill process 4821
[2025-03-10 08:15:03] [WARN]  [disk]    Disk usage at 91% on /dev/sda1
[2025-03-10 08:15:04] [INFO]  [nginx]   Connection accepted from 192.168.1.5
[2025-03-10 08:15:05] [DEBUG] [auth]    Token verified for user: admin
[2025-03-10 08:15:06] [ERROR] [kernel]  Segfault in process 4822 (core dumped)
```

```
[2025-03-10 08:15:07] [WARN]  [nginx]   Retry attempt 3 of 5 for upstream
```

Lines that do not conform to the format must be silently skipped. Blank lines must be silently skipped. The parser must never crash on malformed input.

> ⚠ **Your line parser must handle variable-length MESSAGE fields and SOURCE fields. Hardcoding field widths is incorrect and will be detected during the demo.**

# 5. Multi-Keyword Weighted Scoring

The -k flag accepts a comma-separated list of keywords, for example:

```
./analyzer -k "error,fail,timeout,kill" ...
```

For each log entry, the system counts how many times each keyword appears as a case-sensitive substring in the MESSAGE field (one occurrence = one count even if overlapping). The raw count for each keyword in each entry is then multiplied by a level weight:

| Severity Level | Level Weight |
|---|---|
| ERROR | 4 |
| WARN | 3 |
| INFO | 2 |
| DEBUG | 1 |

The weighted score for a single keyword in a single entry is: raw_count × level_weight. The total weighted score for a keyword across all entries is the sum of all per-entry weighted scores.

Each worker thread must accumulate its weighted scores in thread-local storage (using pthread_key_t) and flush the accumulated values to the shared result structure only at barrier time, under the process-shared mutex.

> ★ **Overlapping keyword occurrences: "errorer" contains "error" twice (at index 0 and index 3). Your search must find all overlapping occurrences. Use a sliding window, not strstr repeated from end of last match.**

The final output must include, for each keyword, a per-level breakdown of the weighted score. See Section 10 for the exact output format.

# 6. Process Types

## 6.1 Parent / Coordinator Process

- Parses all command-line arguments and validates them.
- Reads the configuration file to get the list of log files and the priority source filter file.

- Initializes all shared memory regions (A, B, C, D — see Section 8) and all process-shared primitives.
- Creates one pipe pair (read end, write end) for each Reader Process. These pipes carry progress heartbeat messages from Reader Processes to the parent's Watchdog Thread.
- Forks all child processes in this fixed order: (1) one Reader Process per log file, (2) one Dispatcher Process, (3) four Analyzer Processes (ERROR, WARN, INFO, DEBUG), (4) one Aggregator Process.
- Starts one Watchdog Thread (pthread_create) inside the parent process after all children are forked. The Watchdog Thread uses select() on all Reader pipe read-ends and wakes on a 3-second timeout to print a progress summary to stderr.
- Waits for all children with waitpid in a loop. After all children exit, signals the Watchdog Thread to stop, joins it, prints the final system summary, and releases all resources.

> ⚠ **All child processes must be created exclusively by the parent. No child process may call fork(). The Watchdog Thread runs inside the parent process — it is not a separate process.**

## 6.2 Reader Process

One Reader Process is created for each log file listed in the config file. A Reader Process:

- Spawns T reader threads (T from command line). Reader threads divide the file into T equal-sized byte ranges. A thread whose range boundary falls inside a line must complete that line; the next thread starts at the next newline.
- Reader threads push parsed log_entry_t structs into an internal bounded circular buffer of capacity B, using pthread_mutex_t + pthread_cond_t (no semaphores allowed for the internal buffer).
- A single parser thread consumes from the internal buffer. It classifies each entry by level and pushes it to the Dispatcher's input queue in shared memory region A (process-shared mutex + condvar).
- Each reader thread writes a heartbeat to its dedicated pipe after every 50 lines processed: a single line "[R<i>] <n_lines> lines processed\n". The parent's Watchdog Thread reads these.
- After all reader threads finish and the internal buffer is drained, the parser thread sends one EOF marker per severity level to Region A, then exits.
- The Reader Process then exits normally.

> ℹ **The internal buffer is private to one Reader Process. Use default (non-process-shared) mutex and condvar attributes. Only Region A requires PTHREAD_PROCESS_SHARED attributes.**

## 6.3 Dispatcher Process

Exactly one Dispatcher Process is created. It acts as a routing middleware between Reader Processes and Analyzer Processes:

- Reads entries from Region A (the single input queue fed by all Reader Processes).
- For each entry, checks whether its SOURCE field appears in the priority source list (loaded from the filter file at startup). If yes, copies the entry to Region D (the high-priority buffer) in addition to routing it to the appropriate level buffer in Region B.
- Routes each entry to the correct per-level circular buffer in Region B based on the LEVEL field.

- Forwards EOF markers: when all Reader Processes' EOF markers for a given level have been observed in Region A, the Dispatcher places one EOF marker for that level in Region B.
- Uses pthread_cond_timedwait (with a timeout of T_TIMEOUT seconds, configurable via -T flag, default 10) when waiting for entries in Region A. If the timeout expires and eof_count for all levels equals the number of Reader Processes, the Dispatcher concludes processing and exits.

> **⚠ The Dispatcher is the only process that writes to Region B and Region D. Reader Processes write only to Region A. This single-writer constraint simplifies synchronization — document it in your report.**

### 6.4 Analyzer Process

Four Analyzer Processes are created, one per severity level. Each Analyzer Process:

- Spawns W worker threads (W from command line). Worker threads consume entries from their level's buffer in Region B.
- Each worker thread uses a pthread_key_t destructor to accumulate per-keyword weighted scores in thread-local storage. The destructor flushes the TLS data to the shared result structure when the thread exits.
- After all entries for the level are consumed, every worker thread calls pthread_barrier_wait. After the barrier, only the thread with the lowest system TID (from syscall(SYS_gettid)) writes the level summary to Region C and posts the corresponding semaphore.
- Worker threads also accumulate per-source match counts (which SOURCE had the most hits). The top-3 sources for this level must be included in the Region C result.

> **⚠ You must use gettid() via syscall(SYS_gettid) to identify the reporting thread, not pthread_self(). This will be verified during the demo by inspecting /proc/TID/status.**

### 6.5 Aggregator Process

- Uses pthread_cond_timedwait (T_TIMEOUT second timeout) on a process-shared condvar to wait for each of the four Analyzer results in Region C.
- Once all four results are available, sorts levels by total weighted score descending.
- Writes the human-readable output file (Section 10.1).
- Writes a binary checkpoint file (Section 10.2) using atomic rename: write to a .tmp file, then rename(2) to the final path.
- Prints 'Output files written.' to stdout and exits.

## 7. Watchdog Thread (Inside Parent Process)

After forking all children, the parent spawns exactly one Watchdog Thread using pthread_create. This thread:

- Maintains an array of pipe read-ends (one per Reader Process). Uses select() with a 3-second timeout to wait for data on any pipe.
- When a pipe becomes readable, reads all available heartbeat lines and updates an in-memory progress table (per-reader line count).
- On each 3-second timeout (whether or not any pipe was readable), prints a progress snapshot to stderr:

```
[WATCHDOG] Progress snapshot at T+9s:
  Reader 0 (kernel.log)  : 150 lines processed
  Reader 1 (nginx.log)   :  98 lines processed
  Reader 2 (auth.log)    : 203 lines processed
  Active child processes : 7
```

- The Watchdog Thread terminates when the parent sets a volatile sig_atomic_t shutdown_watchdog flag to 1. The thread checks this flag after each select() call.
- The parent calls pthread_join on the Watchdog Thread before printing the final summary.

> ★ **The Watchdog Thread must never write to stdout — only to stderr. stdout is reserved for the structured output of child processes and the final parent summary.**

# 8. Shared Memory Design

The parent creates four shared memory regions via mmap(MAP_SHARED|MAP_ANONYMOUS) before forking:

### Region A — Dispatcher Input Queue

A single circular buffer of log_entry_t (capacity S_A, from -a flag). Holds all entries produced by all Reader Processes before the Dispatcher consumes them. Contains:

- pthread_mutex_t input_mutex (PTHREAD_PROCESS_SHARED)
- pthread_cond_t not_full_a, not_empty_a (PTHREAD_PROCESS_SHARED)
- int eof_count_per_level[4] — incremented by the parser thread of each Reader Process as it places EOF markers
- int total_readers — set by parent before fork

### Region B — Per-Level Analysis Buffers (×4)

Four independent circular buffers (one per level, capacity S_B each, from -b flag). Written only by the Dispatcher, read only by Analyzer Processes. Each buffer contains:

- pthread_mutex_t level_mutex (PTHREAD_PROCESS_SHARED)
- pthread_cond_t not_full_b, not_empty_b (PTHREAD_PROCESS_SHARED)
- int eof_posted — set to 1 by the Dispatcher when it has forwarded the EOF for this level

### Region C — Results Area

One level_result_t per severity level. Written by the reporting thread of each Analyzer Process. Contains:

```
typedef struct {
    char   level[8];
    long   total_entries;
    double total_weighted_score;
    double per_keyword_score[MAX_KEYWORDS]; /* one per keyword */
```

```
    double per_thread_score[MAX_WORKERS];   /* one per worker  */
    char   top_source[3][64];               /* top-3 sources   */
    long   top_source_hits[3];
    int    ready;
} level_result_t;
```

Region C also contains four sem_t semaphores (one per level) and a process-shared mutex + condvar for the Aggregator's timed wait.

### Region D — High-Priority Buffer

A circular buffer (capacity S_D, from -d flag) receiving copies of entries whose SOURCE appears in the priority filter file. Analyzed by the Aggregator (not by Analyzer Processes). The Aggregator reports the total weighted score for high-priority entries separately in the output.

- pthread_mutex_t priority_mutex (PTHREAD_PROCESS_SHARED)
- pthread_cond_t not_full_d, not_empty_d (PTHREAD_PROCESS_SHARED)
- int dispatcher_done — set by Dispatcher when it finishes

> **⚠ All mutexes and condition variables stored in any shared memory region must be initialized with PTHREAD_PROCESS_SHARED before any child process is forked. Failing to set this attribute causes undefined behavior and will be penalized.**

# 9. Thread-Local Storage Requirement

Each worker thread inside an Analyzer Process must use pthread_key_t to store its per-keyword weighted score accumulator. The required usage pattern is:

- The Analyzer Process calls pthread_key_create once per keyword (or once for the whole array), registering a destructor function.
- Each worker thread calls pthread_setspecific to store a heap-allocated array of doubles (one element per keyword) as its TLS value.
- The destructor is called automatically when the thread exits. The destructor must flush the per-keyword scores to the shared level_result_t in Region C, under the process-shared mutex.
- After the barrier, the reporting thread reads the already-flushed per-thread scores from Region C (not from TLS, which has already been destroyed).

> **⚠ Do not flush TLS data at the barrier — the destructor runs at thread exit, after the barrier. Design your barrier-to-exit sequence accordingly. This is a common mistake that will be specifically tested during the demo.**

# 10. Output File Formats

## 10.1 Human-Readable Output File (-o flag)

```
KEYWORD_LIST: error,fail,timeout
```

```
FILES: 3
TOTAL_WEIGHTED_SCORE: 187.0
HIGH_PRIORITY_SCORE: 52.0

# Levels sorted by total_weighted_score DESC
LEVEL   ENTRIES  WEIGHTED_SCORE   error   fail  timeout
ERROR      41        112.0          28.0   16.0    68.0
WARN       68         53.0          12.0    9.0    32.0
INFO       95         18.0           6.0    4.0     8.0
DEBUG      36          4.0           1.0    2.0     1.0

# Top-3 sources per level
ERROR  kernel:24  nginx:10  auth:7
WARN   disk:18    nginx:15  auth:20
INFO   nginx:42   auth:30   kernel:23
DEBUG  auth:20    disk:10   kernel:6

# Per-thread contributions (weighted score)
ERROR  thread_0:45.0  thread_1:38.0  thread_2:29.0
WARN   thread_0:22.0  thread_1:18.0  thread_2:13.0
INFO   thread_0:8.0   thread_1:6.0   thread_2:4.0
DEBUG  thread_0:2.0   thread_1:1.0   thread_2:1.0
```

Columns in the LEVEL table must be right-aligned and separated by exactly two spaces. All floating-point values must be printed with exactly one decimal place.

## 10.2 Binary Checkpoint File (-O flag)

The binary file must be written to a temporary path (append .tmp) and then atomically renamed to the final path using rename(2). The binary layout is:

```
uint32_t  magic;           /* fixed value: 0xC5E3440B */
uint32_t  version;         /* fixed value: 1          */
uint32_t  num_levels;      /* always 4                */
uint32_t  num_keywords;
double    total_weighted;
double    high_priority_weighted;
level_result_t results[4]; /* in the same order as Region C */
```

The binary file must be written with a single fwrite call for the header and one fwrite per level_result_t. Partial writes must be detected and reported as errors. The file is not human-readable but allows the grader to verify correctness programmatically.

> ⚠ **The binary file must use the native byte order of the machine (no endian conversion required). Do not use fprintf or printf to write the binary file.**

# 11. Command-Line Parameters

| Flag | Parameter | Description | Constraint |
|---|---|---|---|
| -c | <config_file> | Path to config file listing log files (one per line) | Must exist and be readable |
| -f | <filter_file> | Path to priority source filter file (one SOURCE per line) | Must exist and be readable |
| -k | <keywords> | Comma-separated keyword list (no spaces around commas) | 1–MAX_KEYWORDS keywords; MAX_KEYWORDS = 8 |
| -t | <reader_threads> | Reader threads per Reader Process | ≥ 1 |
| -w | <worker_threads> | Worker threads per Analyzer Process | ≥ 1, ≤ MAX_WORKERS = 64 |
| -a | <capacity_A> | Circular buffer capacity for Region A | ≥ 4 |
| -b | <capacity_B> | Circular buffer capacity per level in Region B | ≥ 4 |
| -d | <capacity_D> | Circular buffer capacity for Region D (high-priority) | ≥ 2 |
| -T | <timeout_sec> | Timeout in seconds for timed waits (Dispatcher + Aggregator) | ≥ 1, default 10 |
| -o | <output_file> | Path to human-readable output file | Non-empty |
| -O | <binary_file> | Path to binary checkpoint file | Non-empty |

Example:

```
./analyzer -c services.conf -f priority.txt -k "error,fail,kill" \
           -t 4 -w 3 -a 64 -b 32 -d 16 -T 10 \
           -o results.txt -O results.bin
```

Missing or invalid parameters must produce a descriptive error on stderr and exit with a non-zero code.

# 12. Required Terminal Output

Processes and threads print to stdout. The Watchdog Thread prints to stderr. All lines end with \n. Required messages:

**Parent startup:**

```
[PID:XXXXX] Parent started. Files: 3, Keywords: error,fail,kill
[PID:XXXXX] Shared memory initialized (A:64 B:32x4 D:16).
[PID:XXXXX] Forking Reader 0 -> kernel.log
[PID:XXXXX] Forking Dispatcher
[PID:XXXXX] Forking Analyzer ERROR (index 0)
[PID:XXXXX] Forking Aggregator
[PID:XXXXX] Watchdog thread started.
```

**Reader Process (one per file):**

```
[PID:YYYYY] Reader 0 started. File: kernel.log, Threads: 4
```

```
[PID:YYYYY][TID:TTTTT] Reader thread 0: range [0, 1024) bytes
[PID:YYYYY][TID:TTTTT] Reader thread 0: finished, lines_read=38, malformed=2
[PID:YYYYY] Parser thread: dispatched E:12 W:18 I:35 D:14 -> Region A
[PID:YYYYY] Reader 0 exiting.
```

### Dispatcher Process:

```
[PID:DDDDD] Dispatcher started.
[PID:DDDDD] Routed entry to ERROR buffer. High-priority: YES (source: kernel)
[PID:DDDDD] All EOF markers forwarded to Region B. Exiting.
```

### Analyzer Process (one per level):

```
[PID:ZZZZZ] Analyzer ERROR started. Workers: 3
[PID:ZZZZZ][TID:TTTTT] Worker 0 started.
[PID:ZZZZZ][TID:TTTTT] Worker 0 done. Entries: 17, Weighted score: 45.0
[PID:ZZZZZ][TID:TTTTT] ** Reporting thread (lowest TID). Level: ERROR **
[PID:ZZZZZ][TID:TTTTT]    Total entries: 41 | Total weighted score: 112.0
[PID:ZZZZZ] Analyzer ERROR exiting.
```

### Watchdog Thread (stderr):

```
[WATCHDOG] Progress at T+3s: Reader0=38 Reader1=41 Reader2=0 children_alive=7
[WATCHDOG] Progress at T+6s: Reader0=38 Reader1=95 Reader2=62 children_alive=5
```

### Aggregator Process:

```
[PID:WWWWW] Aggregator started. Waiting for 4 levels...
[PID:WWWWW] ERROR result received.
[PID:WWWWW] All results received. Writing output files...
[PID:WWWWW] Output files written: results.txt, results.bin
[PID:WWWWW] Aggregator exiting.
```

### Parent final summary (after all children exit):

```
==================================================
SYSTEM SUMMARY
Keywords         : error, fail, kill
Log files        : 3
Total entries    : 240
Total weighted   : 187.0
High-priority    : 52.0 (source filter: priority.txt)
  ERROR : 41 entries, score: 112.0
  WARN  : 68 entries, score:  53.0
  INFO  : 95 entries, score:  18.0
  DEBUG : 36 entries, score:   4.0
==================================================
Program terminated successfully.
```

# 13. Required Synchronization Primitives

| Primitive | Where | Purpose |
|---|---|---|

| | | |
|---|---|---|
| pthread_mutex_t (PTHREAD_PROCESS_SHARED) | Region A, B (×4), C, D | Protect all cross-process shared buffers and result areas |
| pthread_cond_t (PTHREAD_PROCESS_SHARED) | Region A, B (×4), D | Block producers when full; block consumers when empty |
| pthread_cond_timedwait | Dispatcher (Region A), Aggregator (Region C) | Timeout-based termination detection |
| pthread_mutex_t (default) | Internal buffer in Reader Process | Protect thread-private buffer (no PROCESS_SHARED needed) |
| pthread_cond_t (default) | Internal buffer in Reader Process | Reader thread / parser thread synchronization |
| pthread_barrier_t | Inside each Analyzer Process | Ensure all W workers finish before any prints results |
| sem_t (process-shared, unnamed) | Region C (×4) | Aggregator blocks until each Analyzer posts completion |
| pthread_key_t | Inside each Analyzer Process | Per-thread keyword score accumulation via TLS |
| volatile sig_atomic_t | Parent process | SIGINT handler → main loop and Watchdog Thread shutdown |
| sigaction | Parent process | Async-safe SIGINT handler installation |

**⚠ Using sem_t for the Reader Process internal buffer (instead of pthread_cond_t) will incur a -20 point penalty. Using pthread_self() instead of gettid() for the reporting thread selection will incur a -15 point penalty. Both will be specifically checked during the demo.**

# 14. Lock Ordering and Deadlock Prevention

You must define and strictly follow a global lock acquisition order. The following order is mandatory:

- 1st: Region A mutex (input_mutex)
- 2nd: Region B level mutex (level_mutex[i]) — only acquire after releasing Region A mutex
- 3rd: Region C result mutex — only acquire after releasing Region B mutex
- 4th: Region D priority mutex — only acquire after releasing all other mutexes

Never acquire a lower-ranked mutex while holding a higher-ranked one. Never call any blocking I/O function while holding any mutex. These rules must be explicitly documented in your report's Synchronization section.

**⚠ Any deadlock observed during testing or the demo results in an automatic -50 point penalty. Design your locking strategy carefully before writing code.**

# 15. Termination

Normal termination chain:

- All Reader Processes exit after placing all entries + EOF markers into Region A.
- Dispatcher forwards all EOFs to Region B and Region D, then exits.
- Analyzer Processes finish consuming Region B, write to Region C, post semaphores, then exit.
- Aggregator waits (with timeout) for all Region C results, writes output files, then exits.
- Parent's main thread returns from the waitpid loop, sets shutdown_watchdog = 1, joins the Watchdog Thread, prints the final summary, and exits.

SIGINT (Ctrl+C) handling:

- Parent's SIGINT handler sets a global volatile sig_atomic_t flag using only async-signal-safe operations.
- Parent's main loop detects the flag, sends SIGTERM to all child process groups, waits for all children with waitpid(-1, ..., WNOHANG) in a loop with a 5-second deadline.
- After all children exit (or the deadline passes), the parent munmaps all shared memory regions, destroys all shared semaphores, joins the Watchdog Thread, and calls _exit(1).
- No zombie processes and no orphan shared memory segments may remain after Ctrl+C.

> **⚠ Do not call printf, malloc, or free inside the SIGINT handler. Use only write(2) and flag setting. Calling async-signal-unsafe functions from a signal handler is undefined behavior and will be tested.**

# 16. Mandatory Test Scenarios

| Test | Command | What to Verify |
|---|---|---|
| 1 – Baseline | ./analyzer -c basic.conf -f filter.txt -k "error" -t 2 -w 2 -a 16 -b 16 -d 8 -o out.txt -O out.bin | Correct counts; human-readable output format; binary file magic number 0xC5E3440B |
| 2 – Multi-keyword | ./analyzer ... -k "error,fail,timeout" | Per-keyword score breakdown correct; weighted scoring verified manually for one entry |
| 3 – Single thread | ./analyzer ... -t 1 -w 1 -a 4 -b 4 -d 2 | Sequential correctness; same totals as Test 1 |
| 4 – High concurrency | ./analyzer ... -t 8 -w 6 -a 128 -b 128 -d 32 (500+ lines) | Results identical across 3 separate runs (determinism check) |
| 5 – No priority matches | Use an empty filter.txt | HIGH_PRIORITY_SCORE: 0.0 in output; no crash |
| 6 – Valgrind | valgrind --leak-check=full --error-exitcode=1 ./analyzer ... | 0 errors, 0 leaks; show full Valgrind output in screenshot |
| 7 – Ctrl+C | Run with large files, kill after 3 sec | No zombies (ps aux \| grep defunct); no leftover shm (ipcs -m) |

⚠ **All screenshots must be uncut, showing the full terminal from the command prompt through program exit. Cropped or edited screenshots will not be accepted.**

# 17. Report Requirements

| Section | Required Content |
|---|---|
| Title Page | Student name, ID, course, HW number |
| Introduction | Problem description, main challenges, your approach |
| System Architecture | Labeled diagram showing all 7 process/thread types and all communication channels (pipes, shared memory regions). Must include data flow arrows. |
| Shared Memory Layout | Struct definitions and memory layout for all four regions (A, B, C, D). Explain which primitive protects which field. |
| Lock Ordering | Explicit table or diagram of lock acquisition order. Proof (argument, not code) that no circular dependency exists. |
| Thread Lifecycle | Full lifecycle of one reader thread and one worker thread: creation → work loop → barrier → TLS destructor → exit. |
| TLS Design | Explain pthread_key_create, pthread_setspecific, and the destructor. Why must the flush happen in the destructor and not at the barrier? |
| Dispatcher Logic | Explain how the Dispatcher handles EOF markers from multiple Reader Processes for multiple levels simultaneously. |
| Termination Chain | Step-by-step trace of normal termination from the last Reader Process exiting to parent final summary. |
| Test Scenarios | All 7 mandatory tests with uncut screenshots and interpretation. |
| Challenges & Solutions | At least 3 concrete problems encountered during development and how each was resolved. |

| Report Penalty | Condition |
|---|---|
| -100 | No report submitted |
| -50 | Report submitted but missing screenshots for mandatory tests |
| -30 | No lock ordering section or diagram |
| -30 | No TLS design explanation |
| -20 | No system architecture diagram or diagram has no labels |

# 18. Grading

**Code — 60 Points**

| Component | Description | Points |
|---|---|---|

| | | |
|---|---|---|
| Process Architecture | Correct fork of all 7 process types; correct waitpid; pipe creation for Watchdog | 10 |
| Reader Process + Internal Buffer | T reader threads with correct byte-range split; mutex+condvar bounded buffer; parser thread; heartbeat pipe writes | 12 |
| Dispatcher Process | Correct routing from Region A to Region B + D; EOF forwarding; pthread_cond_timedwait usage | 10 |
| Analyzer Process + Barrier + TLS | W worker threads; pthread_key_t TLS; pthread_barrier; gettid() reporting thread; per-source top-3 | 14 |
| Aggregator + Binary Output | Timed wait; weighted score sorting; correct text format; binary file with magic + atomic rename | 8 |
| Watchdog Thread + Pipes | select() on pipe read-ends; 3-second stderr snapshots; clean shutdown | 6 |

**Demo — 40 Points**

Individual ~7-minute demo. The TA will ask you to trace code execution, explain design decisions, and run specific commands. Students who cannot explain their own implementation receive 0 on the demo regardless of code quality.

# 19. Penalties

| Condition | Penalty |
|---|---|
| Code does not compile (make fails) | -100 |
| No real multi-process design (fork not meaningfully used) | -50 |
| No real multi-threaded design (pthreads not meaningful) | -50 |
| Internal Reader buffer uses semaphores instead of pthread_cond_t | -20 |
| pthread_self() used instead of gettid() for reporting thread | -15 |
| pthread_key_t / TLS not used in Analyzer worker threads | -20 |
| pthread_cond_timedwait not used in Dispatcher and Aggregator | -15 |
| Atomic rename not used for binary file | -10 |
| Same entry processed by more than one worker | -30 |
| Process-shared mutex/condvar missing PTHREAD_PROCESS_SHARED | -30 |
| Busy waiting / spin-polling anywhere in the program | -40 |
| Deadlock of any kind | -50 |

| | |
|---|---|
| Memory leak (Valgrind) | -30 |
| Zombie processes after exit or Ctrl+C | -30 |
| Orphan shared memory segments after Ctrl+C | -20 |
| Missing or broken Makefile | -20 |
| Each -Wall warning (first 10) | -2 each |
| More than 10 -Wall warnings | -30 |
| Binary output file incorrect magic or structure | -15 |
| Text output file wrong format or wrong sort order | -15 |
| No report or clearly insufficient report | -30 |
| Binary executables in submission archive | -10 |
| Late submission | Not accepted |

# 20. Submission Rules

```
StudentID/
├── Makefile
├── main.c
├── reader.c / reader.h
├── dispatcher.c / dispatcher.h
├── analyzer.c / analyzer.h
├── aggregator.c / aggregator.h
├── shm.c / shm.h
├── watchdog.c / watchdog.h
└── StudentID_report.pdf
```

- Compress as StudentID_HW4.zip. No binaries or .o files.
- Makefile must compile with gcc -Wall -Wextra -pthread and include make clean. Do not auto-run.
- Report: PDF, English.

# 21. Hints and Advice

- Implement and test each stage in isolation: internal buffer first, then Region A, then Region B, then TLS in workers.
- Use MAP_SHARED | MAP_ANONYMOUS with mmap for all shared regions. Calculate the total byte size carefully before calling mmap.
- Initialize all process-shared mutexes and condvars in the parent before any fork. Children inherit the initialized state — never re-initialize in a child.
- pid_t tid = (pid_t)syscall(SYS_gettid); — include <sys/syscall.h> and <unistd.h>. Test that your TID appears in /proc/<TID>/status.
- For overlapping keyword search: use a manual sliding window (index++) not strstr from end of previous match.
- For the binary file: write header first (one fwrite), then loop over 4 level_result_t structs (one fwrite each). Use the return value of fwrite to detect partial writes.

- Compile throughout with -fsanitize=thread to catch races. Disable it for the Valgrind run.
- Test with T=1 W=1 first for correctness, then increase concurrency to expose races.
- Every blocking call (pthread_mutex_lock, sem_wait, pthread_cond_wait, read on pipe) must be checked for error return. Print to stderr and exit on unexpected errors.
- pthread_barrier_destroy, pthread_mutex_destroy, pthread_cond_destroy, pthread_key_delete — call all of these before the relevant process exits.

*For questions, post in the Teams thread below the assignment. Email without posting on Teams first may not receive a timely reply.*